Chương 3

MÔ HÌNH MẠNG NƠ-RON TÍCH CHẬP

Chương 3 sẽ tập trung vào Mạng nơ-ron tích chập (CNN), một trong những kiến trúc mạnh mẽ nhất trong lĩnh vực học sâu, đặc biệt trong xử lý ảnh. Tiếp theo, chúng ta sẽ đi sâu vào các mô hình CNN nổi tiếng như Mạng nơ-ron tích chập LeNet, tìm hiểu về kiến trúc và cách huấn luyện, đánh giá mô hình LeNet-5, AlexNet và Mạng nơ-ron sử dụng khối VGG.

3.1 Giới thiệu về mạng CNN

Một CNN bao gồm một số khối xây dựng cơ bản, được gọi là các lớp CNN. Tại đây, ta tiến hành tìm hiểu các khối kiến trúc này và chức năng của chúng trong kiến trúc CNN. Lưu ý rằng một số lớp này có chức năng cơ bản như chuẩn hóa, tổng hợp, tích chập và các lớp được kết nối đầy đủ. Các lớp cơ bản này được trình bày đầu tiên trong phần này để phát triển hiểu biết cơ bản về các lớp CNN.

3.1.1 Lớp tích chập

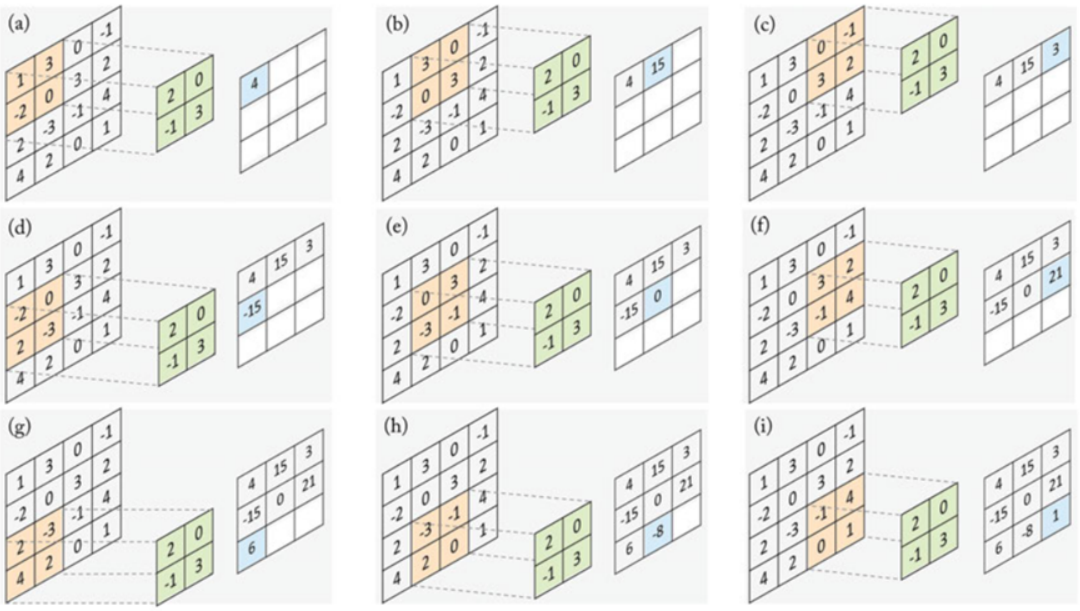
Lớp tích chập là thành phần quan trọng nhất của CNN. Nó bao gồm một tập hợp các bộ lọc (còn được gọi là hạt nhân tích chập) được tích hợp với một đầu vào nhất định để tạo bản đồ đặc trưng đầu ra.

2	0
-1	3

Hình 3.1: Một ví dụ về bộ lọc hình ảnh 2D

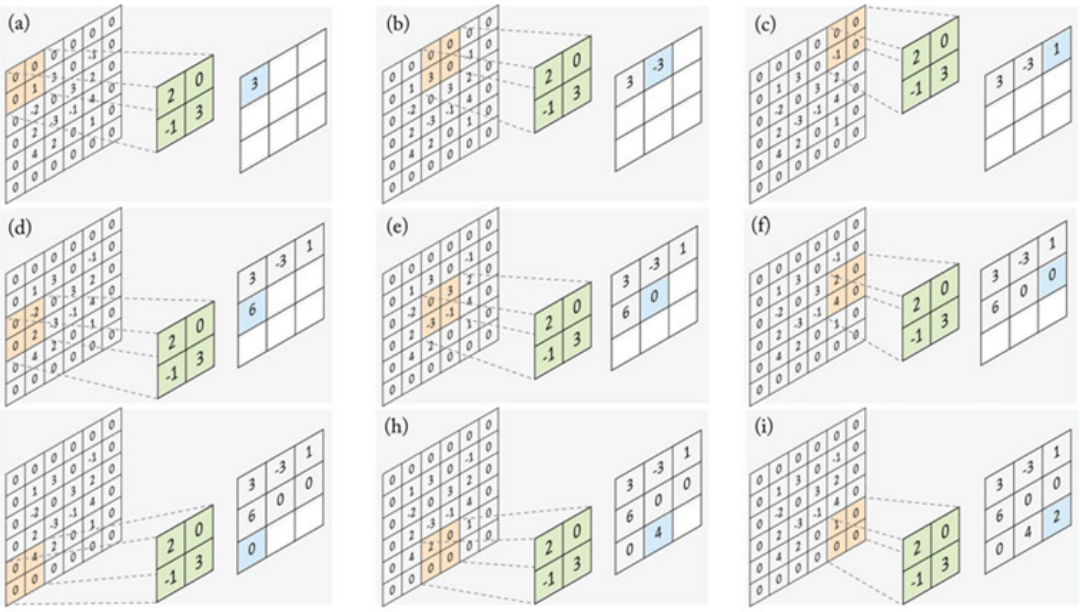
Bộ lọc là gì? Mỗi bộ lọc trong một lớp tích chập là một lưới các số rời rạc. Ví dụ, hãy xem xét bộ lọc 2x2 được hiển thị trong Hình 3.1. Trọng số của mỗi bộ lọc (các số trong lưới) được học trong quá trình đào tạo CNN. Quy trình học tập này liên quan đến việc khởi tạo ngẫu nhiên các trọng số của bộ lọc khi bắt đầu đào tạo. Sau đó, các cặp đầu vào-đầu ra đã cho, các trọng số của bộ lọc được điều chỉnh theo một số lần lặp lại khác nhau trong quy trình học.

Hoạt động tích chập là gì? Chúng ta đã đề cập trước đó rằng lớp tích chập thực hiện tích chập giữa các bộ lọc và đầu vào của lớp. Hãy xem xét một tích chập 2D trong Hình 3.2 để phát triển cái nhìn sâu sắc về hoạt động của lớp. Đưa ra bản đồ đặc trưng đầu vào 2D và bộ lọc tích chập có kích thước ma trận lần lượt là 4x4 và 2x2, lớp tích chập nhân bộ lọc 2x2 với một bản vá được đánh dấu (cũng là 2x2) của bản đồ đặc trưng đầu vào và tính tổng tất cả các giá trị để tạo một giá trị trong bản đồ đặc trưng đầu ra. Lưu ý rằng bộ lọc trượt dọc theo chiều rộng và chiều cao của bản đồ đặc trưng đầu vào và quá trình này tiếp tục cho đến khi bộ lọc không thể trượt thêm nữa.



Hình 3.2: Minh họa hoạt động của một lớp tích chập

Trong ví dụ trên, để tính toán từng giá trị của bản đồ đặc trưng đầu ra, bộ lọc thực hiện bước 1 dọc theo vị trí ngang hoặc dọc (nghĩa là dọc theo cột hoặc hàng của đầu vào). Bước này được gọi là bước sải của bộ lọc tích chập, có thể được đặt thành giá trị khác (hơn 1) nếu cần. Ví dụ, hoạt động tích chập với bước sải 2 được hiển thị trong Hình 3.3. So với bước sải 1 trong ví dụ trước, bước sải 2 dẫn đến bản đồ đặc trưng đầu ra nhỏ hơn. Việc giảm kích thước này được gọi là hoạt động lấy mẫu phụ. Việc giảm kích thước như vậy mang lại sự bất biến vừa phải đối với tỷ lệ và tư thế của các đối tượng, đây là một thuộc tính hữu ích trong các ứng dụng như nhận dạng đối tượng. Chúng ta sẽ thảo luận về các cơ chế lấy mẫu con khác trong phần mà chúng ta thảo luận về các lớp tổng hợp (Pooling layer). Chúng ta đã thấy rằng



Hình 3.3: Minh họa hoạt động của một lớp tích chập với phần đệm bằng 1 và bước sải bằng 2

trong Hình 3.3 kích thước không gian của bản đồ đặc trưng đầu ra bị giảm so với bản đồ đặc trưng đầu vào. Chính xác, đối với bộ lọc có kích thước f x f, bản đồ đặc trưng đầu vào có kích thước h x w và độ dài bước sải s, kích thước đặc trưng đầu ra được đưa ra bởi:

$$h' = \left\lfloor \frac{h - f + s}{s} \right\rfloor, \quad w' = \left\lfloor \frac{w - f + s}{s} \right\rfloor \tag{3.1}$$

Siêu tham số (Hyper-parameters): Các tham số của lớp chập cần được người dùng đặt (dựa trên xác thực

chéo hoặc kinh nghiệm) trước khi học bộ lọc (chẳng hạn như bước sải và phần đệm) được gọi là siêu tham số. Các siêu tham số này có thể được hiểu là các lựa chọn thiết kế cho kiến trúc mạng của chúng ta dựa trên một ứng dụng nhất định.

Trường hợp có kích thước cao (High Dimensional Cases): Trường hợp 2D là trường hợp đơn giản nhất, trong đó bộ lọc chỉ có một kênh duy nhất (được biểu thị dưới dạng ma trận) được kết hợp với các kênh đặc trưng đầu vào để tạo ra phản hồi đầu ra. Đối với các trường hợp có nhiều chiều hơn, ví dụ: khi đầu vào của các lớp CNN là các tensor (ví dụ: khối 3D trong trường hợp biểu diễn thể tích), các bộ lọc cũng là các khối 3D được tích hợp dọc theo chiều cao, chiều rộng và chiều sâu của đặc trưng đầu vào maps để tạo bản đồ đặc trưng đầu ra 3D tương ứng. Tuy nhiên, tất cả các khái niệm mà chúng ta đã thảo luận ở trên cho trường hợp 2D vẫn có thể áp dụng cho việc xử lý đầu vào 3D và các chiều cao hơn (chẳng hạn như học biểu diễn không gian-thời gian 3D). Điểm khác biệt duy nhất là phép tích chập được mở rộng sang một chiều bổ sung, ví dụ: đối với trường hợp 3D, ngoài phép tích chập dọc theo chiều cao và chiều rộng trong trường hợp 2D, phép tích chập cũng được thực hiện dọc theo chiều sâu. Tương tự, đệm không (zero-padding) và bước sải (striding) có thể được thực hiện dọc theo chiều sâu cho trường hợp 3D.

Tuy nhiên, trong một số ứng dụng, chẳng hạn như khử nhiễu hình ảnh, siêu phân giải hoặc phân đoạn, chúng ta muốn giữ kích thước không gian không đổi (hoặc thậm chí lớn hơn) sau khi tích chập. Điều này rất quan trọng vì các ứng dụng này yêu cầu các dự đoán dày đặc hơn ở cấp độ pixel. Hơn nữa, nó cho phép chúng ta thiết kế các mạng sâu hơn (nghĩa là có nhiều lớp trọng số hơn) bằng cách tránh sự sụp đổ nhanh chóng của các kích thước đặc trưng đầu ra. Điều này giúp đạt được hiệu suất tốt hơn và ghi nhãn đầu ra có độ phân giải cao hơn. Điều này có thể đạt được bằng cách áp dụng zero-padding xung quanh bản đồ đặc trưng đầu vào.

Như được hiển thị trong Hình 1.20, đệm không (zero-padding) vào chiều ngang và dọc cho phép chúng ta tăng kích thước đầu ra và do đó mang lại sự linh hoạt hơn trong thiết kế kiến trúc. Ý tưởng cơ bản là tăng kích thước của bản đồ đặc trưng đầu vào sao cho thu được bản đồ đặc trưng đầu ra, với các kích thước mong muốn. Nếu p biểu thị mức tăng trong bản đồ đặc trưng đầu vào dọc theo mỗi thứ nguyên (bằng cách đệm các số 0), thì chúng ta có thể biểu thị các thứ nguyên bản đồ đặc trưng đầu ra đã sửa đổi như sau:

3.1.2 Lớp Pooling

Một lớp gộp hoạt động trên các khối của bản đồ đặc trưng đầu vào và kết hợp các kích hoạt đặc trưng. Thao tác kết hợp này được xác định bởi một hàm tổng hợp chẳng hạn như hàm trung bình hoặc hàm tối đa. Tương tự như lớp tích chập, chúng ta cần chỉ định kích thước của vùng gộp và bước sải. Hình 1.20 cho thấy hoạt động gộp tối đa (max pooling), trong đó kích hoạt tối đa được chọn từ khối giá trị đã chọn. Cửa sổ này được trượt trên các bản đồ đặc trưng đầu vào với kích thước bước được xác định bởi bước sải (1 trong trường hợp của Hình 1.20). Nếu kích thước của vùng gộp được cung cấp bởi f x f, với một bước sải s, kích thước của bản đồ đặc trưng đầu ra được cung cấp bởi:

$$h' = \left\lfloor \frac{h - f + s}{s} \right\rfloor, \quad w' = \left\lfloor \frac{w - f + s}{s} \right\rfloor \tag{3.2}$$

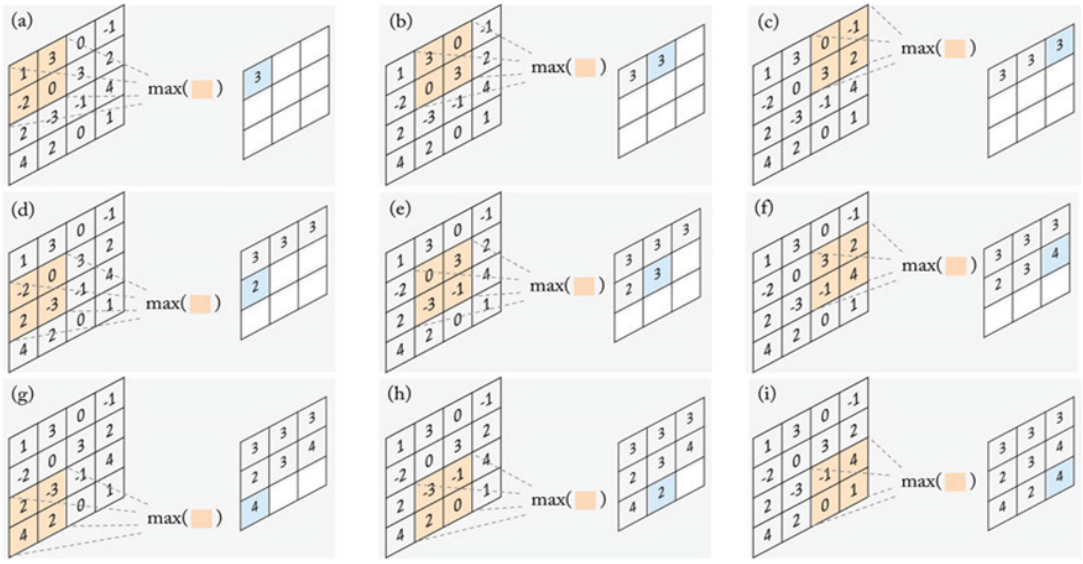
Hoạt động gộp (pooling) giảm kích thước của bản đồ đặc trưng đầu vào một cách hiệu quả. Quá trình lấy mẫu xuống như vậy rất hữu ích để có được một biểu diễn đặc trưng nhỏ gọn, bất biến đối với những thay đổi vừa phải về tỷ lệ đối tượng, tư thế và bản dịch trong một hình ảnh.

3.1.3 Lớp Fully Connected (FC)

Các lớp được kết nối đầy đủ về cơ bản tương ứng với các lớp tích chập với các bộ lọc có kích thước 1x1. Mỗi đơn vị trong một lớp được kết nối đầy đủ được kết nối dày đặc với tất cả các đơn vị của lớp trước đó. Trong một CNN điển hình, các lớp được kết nối đầy đủ thường được đặt ở cuối kiến trúc. Hoạt động của nó có thể được biểu diễn dưới dạng phép nhân ma trận đơn giản, sau đó thêm một vectơ các số hạng sai lệch (bias) và áp dụng một hàm phi tuyến tính theo phần tử:

$$y = f\left(W^T x + b\right) \tag{3.3}$$

Trong đó x và y lần lượt là vectơ kích hoạt đầu vào và đầu ra, W biểu thị ma trận chứa trọng số của các kết nối giữa các đơn vị lớp và b biểu thị vectơ số hạng sai lệch.



Hình 3.4: Minh họa hoạt động của một lớp gộp tối đa (max-pooling layer) với vùng gộp là 2x2 và bước sải bằng 1

3.2 Mạng nơ-ron tích chập LeNet

3.2.1 Giới thiệu

LeNet, một trong những mô hình mạng nơ-ron tích chập (Convolutional Neural Network - CNN) đầu tiên, đã đóng vai trò quan trọng trong sự phát triển của học sâu (deep learning) và có ảnh hưởng sâu rộng trong lĩnh vực nhận dạng hình ảnh. Được phát triển bởi Yann LeCun [1] cùng các cộng sự vào đầu những năm 1990, LeNet không chỉ là một bước đột phá về mặt kỹ thuật mà còn là minh chứng cho sức mạnh của các mô hình học sâu trong việc giải quyết các bài toán thực tế phức tạp, như nhận dạng chữ viết tay, mà trước đó chỉ có thể giải quyết bằng các phương pháp học máy truyền thống.

LeNet đã đánh dấu một bước chuyển mình mạnh mẽ trong việc áp dụng học máy, đặc biệt là học sâu, vào nhận dạng hình ảnh. Trước khi LeNet xuất hiện, các mô hình phân loại hình ảnh chủ yếu dựa vào các kỹ thuật trích xuất đặc trưng thủ công, kết hợp với các thuật toán phân loại như máy vector hỗ trợ (SVM) để phân loại đối tượng trong ảnh. Tuy nhiên, LeNet đã thay đổi cách tiếp cận này bằng cách sử dụng các mạng nơ-ron tích chập (CNNs) để tự động học các đặc trưng từ dữ liệu mà không cần đến sự can thiệp thủ công. Điều này mở ra một kỷ nguyên mới trong nhận dạng hình ảnh, nơi các mô hình không chỉ có thể học từ dữ liệu mà còn có thể tự động điều chỉnh và tối ưu hóa các tham số của chúng để cải thiện hiệu suất.

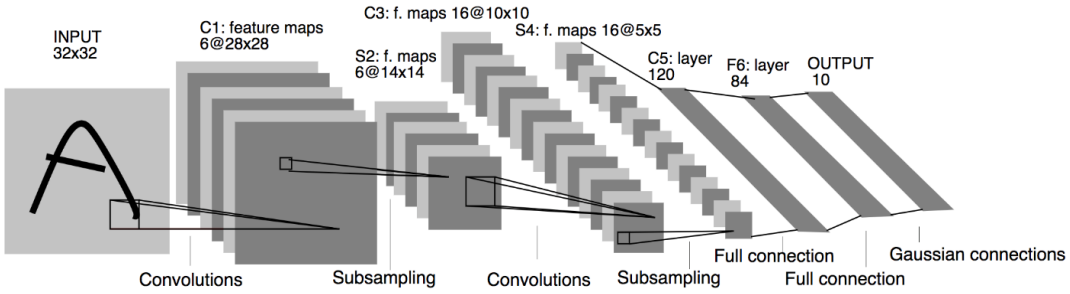
Kiến trúc của LeNet rất đơn giản nhưng lại cực kỳ hiệu quả. Mạng LeNet-5, phiên bản nổi bật nhất của mô hình này, được xây dựng từ một chuỗi các lớp nơ-ron được tổ chức theo một cấu trúc đặc biệt. Mô hình này bao gồm các lớp tích chập (convolutional layers), các lớp gộp (pooling layers), và các lớp kết nối đầy đủ (fully connected layers). Mỗi lớp tích chập trong LeNet sử dụng các bộ lọc nhỏ để quét qua hình ảnh đầu vào và trích xuất các đặc trưng không gian quan trọng như các cạnh, góc, hoặc hình dạng. Các lớp gộp làm nhiệm vụ giảm kích thước không gian của dữ liệu, từ đó giúp giảm thiểu số lượng tham số và ngăn ngừa hiện tượng overfitting, một trong những vấn đề lớn khi huấn luyện các mô hình phức tạp. Cuối cùng, các lớp kết nối đầy đủ thực hiện nhiệm vụ phân loại dựa trên các đặc trưng đã học, giúp mô hình đưa ra kết quả chính xác cho bài toán nhận dạng.

Một trong những điểm nổi bật nhất của LeNet là khả năng tự động học các đặc trưng của hình ảnh mà không cần sự can thiệp thủ công, điều này hoàn toàn khác biệt so với các phương pháp trước đây. Các lớp tích chập trong LeNet có khả năng học được từ các đặc trưng cơ bản như cạnh, góc, hay các đường viền cho đến các đặc trưng phức tạp hơn như hình dạng và cấu trúc của các đối tượng trong ảnh. Chính điều này giúp LeNet có thể nhận diện các đối tượng trong hình ảnh một cách tự động và chính xác, vượt trội hơn nhiều so với các phương pháp phân loại truyền thống. Mô hình này đã mở rộng ứng dụng của mạng nơ-ron tích chập vào nhiều lĩnh vực khác nhau, chẳng hạn như nhận dạng chữ viết tay, nhận diện khuôn mặt, phân loại đối tượng trong ảnh và video, nhận dạng hành động, và thậm chí trong các hệ thống y tế để phân tích hình ảnh y khoa.

LeNet, mặc dù không còn là mô hình tiên tiến nhất trong thời đại hiện nay, nhưng vẫn đóng vai trò là nền tảng cho sự phát triển của các mạng nơ-ron tích chập hiện đại. Hơn nữa, kiến trúc của LeNet đã

và đang ảnh hưởng lớn đến các nghiên cứu sau này, góp phần mở rộng ứng dụng của học sâu trong các lĩnh vực như nhận dạng hình ảnh, thị giác máy tính, robot, y học, và nhiều ngành công nghiệp khác. Việc hiểu rõ và nghiên cứu LeNet không chỉ giúp nắm vững những nguyên lý cơ bản của mạng nơ-ron mà còn mở ra những cơ hội mới trong việc thiết kế các mô hình học sâu ngày càng mạnh mẽ và hiệu quả hơn.

3.2.2 Kiến trúc LeNet



Hình 3.5: Kiến trúc tổng quan của LeNet

Kiến trúc của LeNet-5 bao gồm 7 lớp, với sự kết hợp của các lớp tích chập, lớp gộp và lớp kết nối đầy đủ, được tổ chức như sau:

- Lớp 1: Lớp Tích Chập (Convolutional Layer 1)

Lớp tích chập đầu tiên sử dụng 6 bộ lọc kích thước 5x5, tạo ra 6 bản đồ đặc trưng (feature maps) có kích thước 28x28. Các bộ lọc này thực hiện phép tích chập trên hình ảnh đầu vào, giúp trích xuất các đặc trưng cơ bản như các cạnh, góc, và các đối tượng đơn giản. Lớp này giúp mạng bắt đầu nhận diện các đặc trưng không gian cơ bản trong ảnh.

- Lớp 2: Lớp Gộp (Subsampling or Pooling Layer 1)

Lớp gộp này thực hiện phép giảm kích thước dữ liệu bằng cách sử dụng các phép gộp trung bình (average pooling) hoặc gộp cực trị (max pooling). Kết quả là kích thước của các bản đồ đặc trưng được giảm từ 28x28 xuống 14x14, giúp giảm số lượng tham số và ngăn ngừa overfitting. Mặc dù giảm kích thước, lớp này vẫn giữ lại các đặc trưng quan trọng từ lớp tích chập trước.

- Lớp 3: Lớp Tích Chập (Convolutional Layer 2)

Lớp tích chập thứ hai sử dụng 16 bộ lọc kích thước 5x5 để quét qua các bản đồ đặc trưng từ lớp gộp, tạo ra 16 bản đồ đặc trưng mới có kích thước 10x10. Lớp này học các đặc trưng phức tạp hơn, như các hình dạng và cấu trúc trong ảnh, dựa trên các đặc trưng cơ bản đã được học từ lớp tích chập đầu tiên.

- Lớp 4: Lớp Gộp (Subsampling or Pooling Layer 2)

Lớp gộp thứ hai tiếp tục giảm kích thước các bản đồ đặc trưng từ 10x10 xuống 5x5, giúp giảm thêm số lượng tham số và cải thiện khả năng tổng quát của mô hình.

- Lớp 5: Lớp Kết Nối Đầy Đủ (Fully Connected Layer 1)

Sau khi các đặc trưng đã được trích xuất và giảm kích thước qua các lớp tích chập và gộp, lớp kết nối đầy đủ đầu tiên bắt đầu kết nối các đặc trưng này lại với nhau để thực hiện phân loại. Lớp này có 120 Node, mỗi nút kết nối với tất cả các nút trong lớp trước, giúp học các mối quan hệ phức tạp giữa các đặc trưng. - Lớp 6: Lớp Kết Nối Đầy Đủ (Fully Connected Layer 2)

Lớp kết nối đầy đủ thứ hai có 84 Nodes, tiếp tục xử lý các đặc trưng đã được học và chuẩn bị cho lớp đầu ra.

- Lớp 7: Lớp Đầu Ra (Output Layer)

Cuối cùng, lớp đầu ra của LeNet có 10 nút, mỗi nút đại diện cho một lớp phân loại trong bộ dữ liệu. Đối với bài toán nhận dạng chữ viết tay MNIST, mỗi nút đại diện cho một chữ số từ 0 đến 9. Lớp này sử dụng hàm kích hoạt softmax để tính toán xác suất cho mỗi lớp, từ đó đưa ra dự đoán cuối cùng của mô hình.

3.2.3 Huấn luyện và đánh giá mô hình LeNet-5

a. Chuẩn bị dữ liệu MNIST

```
# Import necessary library
import numpy as np
import seaborn as sns
import tensorflow as tf
from keras import datasets, layers, models, Sequential
from keras.layers import Dense, Activation, Dropout, Input, Conv2D, MaxPooling2D,
Flatten, ReLU, AveragePooling2D
from keras.losses import SparseCategoricalCrossentropy
import matplotlib.pyplot as plt
from keras.utils import plot_model
from tensorflow.keras import utils
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Load dataset MNIST
(train_images, train_labels), (test_images, test_labels) = datasets.mnist.load_data()

# Print the size of the MNIST train and test sets
print(train_images.shape)
print(train_labels.shape)
print(test_images.shape)
print(test_labels.shape)
```

Trong bài này sẽ thực hiện huấn luyện mô hình LeNet với tập dữ liệu MNIST. Bộ dữ liệu MNIST (Modified National Institute of Standards and Technology) là một bộ dữ liệu nổi tiếng trong lĩnh vực học máy và thị giác máy tính, bao gồm 60,000 hình ảnh huấn luyện và 10,000 hình ảnh kiểm tra của các chữ số viết tay từ 0 đến 9. Mỗi hình ảnh có kích thước 28x28 pixel, đơn sắc, với giá trị điểm ảnh từ 0 đến 255. Dữ liệu này được sử dụng chủ yếu để thử nghiệm các mô hình phân loại hình ảnh, đặc biệt là trong nhận dạng chữ số viết tay.

b. Xây dựng mô hình

```
model1 = Sequential()
# First Convolutional Layer
# Input = (32, 32, 1)
# Output = [(32 + 0 - 5) / 1] + 1 = (28, 28, 6)
model1.add(Conv2D(filters=6, kernel_size=(5,5), activation='relu',
input_shape=(32, 32, 1), name='Conv1'))

# Input = (28, 28, 6)
# Output = [(28 + 0 - 2) / 2] + 1 = (14, 14, 6)
model1.add(AveragePooling2D(pool_size=(2, 2), name='AvgPool1'))

# Second Convolutional Layer
# Input = (14, 14, 6)
# Output = [(14 + 0 - 5) / 1] + 1 = (10, 10, 16)
model1.add(Conv2D(filters=16, kernel_size=(5,5), activation='relu', name='Conv2'))

# Input = (10, 10, 16)
# Output = [(10 + 0 - 2) / 2] + 1 = (5, 5, 16)
model1.add(AveragePooling2D(pool_size=(2, 2), name='AvgPool2'))

# Input = (5, 5, 16)
# Output = 5 * 5 * 16 = 400
model1.add(Flatten())                          # Input Layer of neural network
model1.add(Dense(120, activation='relu'))      # Hidden Layer (Fully Connected Layer 1)
model1.add(Dense(84, activation='relu'))       # Hidden Layer (Fully Connected Layer 2)
model1.add(Dense(10, activation='softmax'))    # Output Layer
```

Đoạn code trên thực hiện việc xây dựng một mô hình mạng nơ-ron tích chập (CNN) đơn giản theo kiến trúc LeNet-5, được triển khai bằng thư viện Keras. Mô hình này gồm các lớp tích chập, lớp gộp, và các lớp kết nối đầy đủ, được thiết kế để nhận dạng các hình ảnh có kích thước đầu vào 32x32x1.

- Lớp Tích Chập Đầu Tiên (Conv2D): Mô hình bắt đầu với một lớp tích chập có 6 bộ lọc (filters) với kích thước 5x5, sử dụng hàm kích hoạt ReLU, giúp trích xuất các đặc trưng cơ bản từ ảnh đầu vào. Kết quả đầu ra của lớp này là các bản đồ đặc trưng có kích thước 28x28 với 6 kênh (feature maps).

- Lớp Gộp (AveragePooling2D): Sau lớp tích chập, một lớp gộp trung bình (average pooling) được áp dụng với kích thước pool 2x2, giúp giảm kích thước của các bản đồ đặc trưng từ 28x28 xuống còn 14x14 mà vẫn giữ lại các đặc trưng quan trọng.

- Lớp Tích Chập Thứ Hai (Conv2D): Mô hình tiếp tục với lớp tích chập thứ hai có 16 bộ lọc với kích thước 5x5, sử dụng hàm kích hoạt ReLU. Lớp này giúp mạng học các đặc trưng phức tạp hơn từ các bản đồ đặc trưng được tạo ra từ lớp gộp trước đó. Kết quả đầu ra có kích thước 10x10 và 16 kênh.

- Lớp Gộp Thứ Hai (AveragePooling2D): Tiếp theo, một lớp gộp trung bình thứ hai được áp dụng để giảm kích thước bản đồ đặc trưng từ 10x10 xuống còn 5x5, tiếp tục giúp giảm số lượng tham số và ngăn ngừa hiện tượng overfitting.

- Lớp Flatten và Các Lớp Kết Nối Đầy Đủ (Dense): Sau khi các đặc trưng đã được trích xuất qua các lớp tích chập và gộp, mô hình sử dụng lớp Flatten để chuyển đổi các bản đồ đặc trưng 5x5 thành một vector 1 chiều. Sau đó, mô hình có hai lớp kết nối đầy đủ với 120 và 84 nút, mỗi lớp sử dụng hàm kích hoạt ReLU, giúp học các mối quan hệ phức tạp giữa các đặc trưng. Cuối cùng, lớp đầu ra có 10 nút với hàm kích hoạt softmax để phân loại hình ảnh vào một trong 10 lớp (0 đến 9).

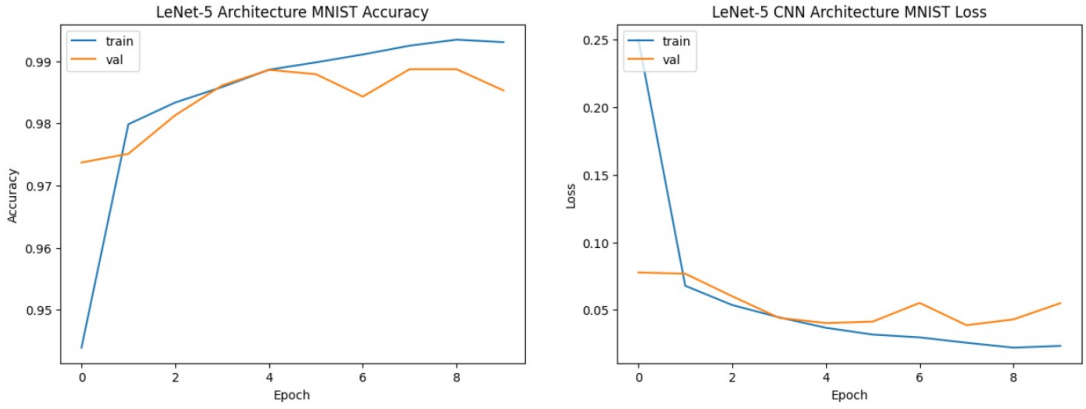
c. Huấn luyện mô hình

```
# compile model
model1.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
# train model
model1_history = model1.fit(train_images, train_labels,
validation_data=(test_images, test_labels), epochs=10)
```

Đoạn code trên thực hiện việc biên dịch và huấn luyện mô hình LeNet sử dụng bộ tối ưu hóa Adam, hàm mất mát Categorical Crossentropy và đo lường độ chính xác (accuracy) trong quá trình huấn luyện. - Biên dịch mô hình (compile): Mô hình được biên dịch với bộ tối ưu hóa Adam, một thuật toán phổ biến và hiệu quả trong huấn luyện mạng nơ-ron, giúp cập nhật các trọng số của mô hình trong quá trình huấn luyện. Hàm mất mát Categorical Crossentropy được sử dụng cho các bài toán phân loại đa lớp, trong đó mỗi mẫu dữ liệu thuộc về một lớp cụ thể. Các chỉ số metrics=['accuracy'] được chỉ định để theo dõi độ chính xác của mô hình trong quá trình huấn luyện và kiểm tra.

- Huấn luyện mô hình (fit): Mô hình sau đó được huấn luyện bằng cách sử dụng tập dữ liệu huấn luyện train_images và nhãn train_labels, với tập kiểm thử test_images và test_labels được sử dụng để xác nhận hiệu suất của mô hình trong mỗi epoch. Quá trình huấn luyện sẽ diễn ra trong 10 epoch, nghĩa là mô hình sẽ được huấn luyện qua 10 lần quét toàn bộ dữ liệu huấn luyện. Hàm fit trả về một đối tượng model1_history, chứa thông tin về quá trình huấn luyện, bao gồm độ chính xác và mất mát trên cả tập huấn luyện và tập kiểm tra trong mỗi epoch, giúp theo dõi sự cải thiện của mô hình qua từng lần huấn luyện.

d. Đánh giá mô hình



Hình 3.6: Quá trình huấn luyện mô hình

Từ hai biểu đồ trên, có thể nhận thấy rằng độ lỗi của mô hình LeNet-5 tiến dần về 0 qua từng epoch, cho thấy mô hình đang dần học cách dự đoán chính xác hơn. Đây là một dấu hiệu tích cực, phản ánh sự tiến bộ rõ rệt của mô hình trong việc học từ dữ liệu huấn luyện và cải thiện hiệu suất trên dữ liệu xác thực.

Hơn nữa, hiệu suất của mô hình, được đo bằng độ chính xác, dần tiệm cận giá trị 1 trên cả tập huấn luyện và tập xác thực. Đặc biệt, trong những epoch cuối, độ chính xác trên hai tập này gần như đồng đều, cho thấy mô hình không chỉ học tốt mà còn tránh được hiện tượng overfitting – một yếu tố quan trọng trong việc xây dựng một mô hình học máy hiệu quả.

Xét về độ ổn định của hiệu suất (độ chính xác) và hàm mất mát, cả hai biểu đồ đều minh chứng rõ ràng cho khả năng tổng quát hóa tốt của mô hình. Độ chênh lệch nhỏ giữa tập huấn luyện và tập xác thực cho thấy mô hình không chỉ hoạt động tốt trên dữ liệu đã học mà còn xử lý tốt dữ liệu chưa từng

thấy. Điều này khẳng định rằng LeNet-5 là một kiến trúc phù hợp và hiệu quả khi áp dụng trên tập dữ liệu MNIST.

3.3 Mạng nơ-ron tích chập sâu (AlexNet)

3.3.1 Giới thiệu

Mặc dù đã trở nên nổi tiếng trong cộng đồng thị giác máy tính và học máy sau khi LeNet được giới thiệu, tuy nhiên, những hạn chế của LeNet dần trở nên rõ ràng khi đối mặt với các bài toán phức tạp hơn, đặc biệt là khi làm việc với các tập dữ liệu lớn và đa dạng hơn. Một trong những yếu tố chính khiến LeNet không thể mở rộng quy mô để xử lý dữ liệu thực tế là sự thiếu hụt tài nguyên tính toán mạnh mẽ vào thời điểm đó. Các máy tính với phần cứng hạn chế, đặc biệt là thiếu GPU, không thể đáp ứng được nhu cầu tính toán phức tạp của các mô hình mạng nơ-ron sâu khi đối mặt với các tập dữ liệu quy mô lớn.

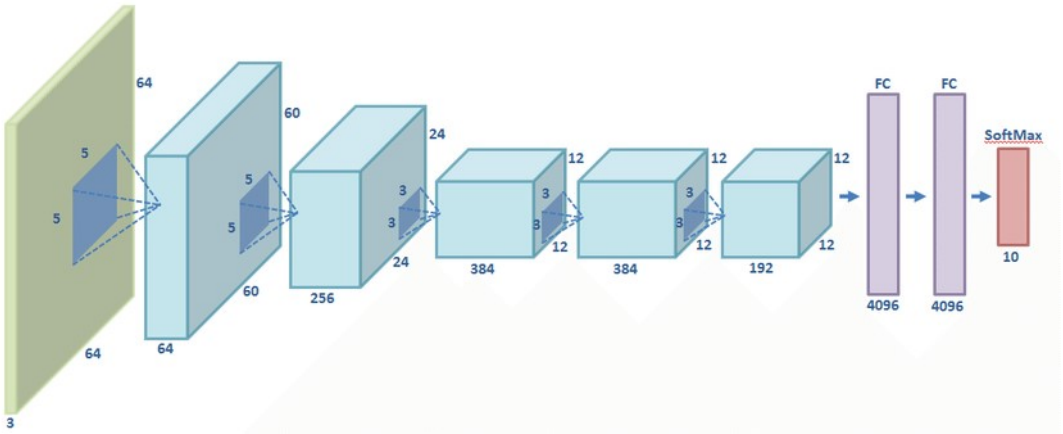
Trong suốt gần hai thập kỷ từ những năm 1990 cho đến năm 2012, mặc dù tiềm năng lý thuyết của CNN luôn được cộng đồng học thuật công nhận, nhưng mạng nơ-ron tích chập không thể đạt được sự đột phá thực sự trong các ứng dụng thực tế. Các phương pháp học máy truyền thống vẫn thống trị, và CNN không thể hiện được ưu thế trong bối cảnh thiếu hụt tài nguyên và dữ liệu. Tuy nhiên, vào năm 2012, sự xuất hiện của AlexNet đã đánh dấu một bước ngoặt quan trọng trong lịch sử phát triển của mạng nơ-ron tích chập. Với kiến trúc mạng mạnh mẽ và khả năng khai thác triệt để các tài nguyên tính toán, AlexNet đã vượt qua những rào cản mà các mô hình trước đó không thể vượt qua.

AlexNet không chỉ là một sự cải tiến của LeNet mà còn là một biến thể sáng tạo, kết hợp các kỹ thuật tiên tiến nhằm giải quyết những vấn đề tồn tại từ trước. Một trong những cải tiến quan trọng nhất của AlexNet là việc tận dụng sức mạnh tính toán của GPU, điều này giúp giảm thiểu thời gian huấn luyện mô hình, từ đó có thể xử lý được các tập dữ liệu lớn hơn rất nhiều so với trước đây. Ngoài ra, việc áp dụng tầng dropout để giảm thiểu hiện tượng overfitting đã giúp AlexNet có khả năng huấn luyện ổn định hơn khi làm việc với những tập dữ liệu có sự phức tạp cao. Hơn nữa, kiến trúc của AlexNet được thiết kế để có thể xử lý hàng triệu hình ảnh, với độ phân giải cao và độ phức tạp lớn, điều mà các mô hình trước đó không thể đáp ứng. Chính những cải tiến này đã giúp AlexNet không chỉ vượt qua các phương pháp học máy truyền thống mà còn trở thành một trong những kiến trúc CNN nổi bật nhất, đặt nền móng cho sự bùng nổ của học sâu trong thập kỷ tiếp theo.

AlexNet không chỉ đơn thuần mang tính đột phá về mặt kỹ thuật mà còn mang ý nghĩa biểu tượng trong sự phát triển của lĩnh vực học sâu. Nó chứng minh rằng khi công nghệ tính toán và dữ liệu được phát triển đầy đủ, mạng nơ-ron tích chập có thể đạt được những kết quả ấn tượng trong các bài toán thị giác máy tính phức tạp. Thành công vượt trội của AlexNet trong cuộc thi ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012 [2] không chỉ khẳng định hiệu quả của CNN mà còn chỉ ra rằng mạng nơ-ron tích chập có thể được áp dụng vào các lĩnh vực phức tạp khác ngoài nhận dạng hình ảnh, như phát hiện đối tượng, nhận diện khuôn mặt, phân loại các thể loại ảnh phức tạp và nhiều ứng dụng khác trong y học, robot, hay thậm chí là xử lý ngôn ngữ tự nhiên.

Thành công của AlexNet đã truyền cảm hứng cho sự phát triển của những kiến trúc mạng tiên tiến hơn, bao gồm VGG, ResNet, Inception và nhiều mô hình khác, những mô hình này tiếp tục mở rộng khả năng của học sâu trong các bài toán thị giác máy tính và các lĩnh vực liên ngành. Các kỹ thuật tối ưu hóa, như giảm thiểu hiện tượng overfitting, tăng cường dữ liệu, hay sử dụng các tầng convolution sâu hơn, đã giúp các mô hình kế tiếp vượt qua các rào cản trước đây và mở rộng khả năng ứng dụng của mạng nơ-ron tích chập vào các bài toán mới, đầy thử thách. Từ đó, AlexNet không chỉ thúc đẩy sự phát triển mạnh mẽ của học sâu trong lĩnh vực thị giác máy tính mà còn góp phần định hình lại toàn bộ xu hướng nghiên cứu và ứng dụng trí tuệ nhân tạo hiện đại.

3.3.2 Kiến trúc AlexNet



Hình 3.7: Kiến trúc mạng AlexNet

AlexNet, một mô hình tiên phong trong học sâu, được thiết kế với các tầng tích chập đầu tiên sử dụng cửa sổ tích chập có kích thước lớn 11×11. Kích thước này phù hợp với đặc điểm của các hình ảnh trong tập dữ liệu ImageNet, nơi chiều cao và chiều rộng ảnh thường lớn gấp nhiều lần so với MNIST, và các vật thể thường chiếm nhiều điểm ảnh hơn. Trong các tầng tiếp theo, kích thước cửa sổ tích chập được giảm dần, từ 5×5 ở tầng thứ hai xuống còn 3×3 ở các tầng sau, giúp tăng khả năng trích xuất đặc trưng chi tiết.

Các tầng tích chập thứ nhất, thứ hai và thứ năm đều được theo sau bởi các tầng gộp cực đại với cửa sổ kích thước 3×3 và sải bước là 2, giúp giảm độ phức tạp và tăng tính hiệu quả trong việc phát hiện đặc trưng. Đặc biệt, số lượng kênh tích chập của AlexNet vượt xa LeNet, nhiều hơn đến mười lần, mang lại khả năng học tốt hơn.

Sau các tầng tích chập là hai tầng kết nối đầy đủ với 4096 đầu ra, tạo nên một lượng tham số khổng lồ lên đến gần 1 GB. Do hạn chế bộ nhớ của các GPU thế hệ trước, phiên bản gốc của AlexNet sử dụng thiết kế hai luồng dữ liệu song song trên hai GPU, mỗi GPU xử lý một nửa mô hình. Tuy nhiên, với sự tiến bộ của công nghệ, các GPU hiện nay có bộ nhớ dồi dào, cho phép ta triển khai toàn bộ mô hình trên một GPU mà không gặp trở ngại nào. Điều này làm cho phiên bản hiện tại của AlexNet đơn giản hơn và hiệu quả hơn so với thiết kế ban đầu.

3.3.3 Huấn luyện và đánh giá mô hình AlexNet

a. Chuẩn bị dữ liệu MNIST Fashion

```
# Import necessary library
import numpy as np
import seaborn as sns
import tensorflow as tf
from keras import datasets, layers, models, Sequential
from keras.layers import Dense, Activation, Dropout, Input, Conv2D, MaxPooling2D,
Flatten, ReLU, AveragePooling2D
from keras.losses import SparseCategoricalCrossentropy
import matplotlib.pyplot as plt
from keras.utils import plot_model
from tensorflow.keras import utils
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Load model Fashion-MNIST
(x_train,y_train),(x_test,y_test) = datasets.fashion_mnist.load_data()

# Print size of train and test datasets
print(x_train.shape)
print(y_train.shape)
print(x_test.shape)
print(y_test.shape)
```

Trong bài này sẽ huấn luyện mô hình AlexNet với tập dữ liệu MNIST Fashion. Bộ dữ liệu MNIST Fashion là một phiên bản mở rộng của MNIST, bao gồm 60.000 hình ảnh huấn luyện và 10.000 hình ảnh kiểm tra của các mặt hàng thời trang như áo thun, quần, giày dép, túi xách, v.v. Mỗi hình ảnh có kích thước

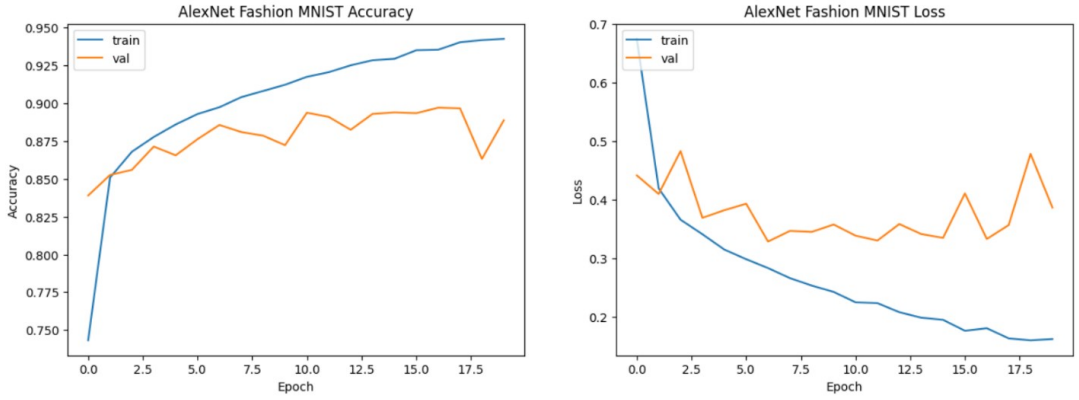
28x28 pixel, đơn sắc, với giá trị điểm ảnh từ 0 đến 255. Dữ liệu này thường được sử dụng để thử nghiệm các mô hình phân loại hình ảnh, đặc biệt trong các bài toán liên quan đến nhận diện và phân loại sản phẩm thời trang.

b. Huấn luyện mô hình

```
# Compile model
AlexNet_model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
# Train model
AlexNet = AlexNet_model.fit(x_train, y_train, epochs=20, validation_data=(x_test, y_test))
```

Đoạn code trên thực hiện việc biên dịch và huấn luyện mô hình AlexNet với bộ tối ưu hóa Adam, hàm mất mát Categorical Crossentropy, và đo lường độ chính xác (accuracy) trong quá trình huấn luyện. Mô hình được biên dịch với bộ tối ưu hóa Adam, một thuật toán phổ biến và hiệu quả để cập nhật trọng số của mạng nơ-ron, cùng hàm mất mát Categorical Crossentropy, phù hợp cho bài toán phân loại nhiều lớp. Chỉ số đo lường accuracy được sử dụng để theo dõi hiệu suất của mô hình trong suốt quá trình huấn luyện và kiểm tra. Sau đó, mô hình được huấn luyện trên dữ liệu từ train_generator và kiểm tra hiệu suất trên tập val_generator trong 30 epoch. Hàm fit trả về một đối tượng lịch sử (history), chứa thông tin chi tiết về độ chính xác và mất mát trên cả tập huấn luyện và tập xác thực trong từng epoch, giúp theo dõi sự cải thiện của mô hình qua từng chu kỳ huấn luyện.

c. Đánh giá mô hình



Hình 3.8: Quá trình huấn luyện mô hình AlexNet

Đầu tiên, biểu đồ hàm mất mát (Loss) cho thấy rằng giá trị mất mát trên tập huấn luyện (train) giảm dần qua từng epoch, minh chứng cho việc mô hình ngày càng học tốt hơn từ dữ liệu. Tuy nhiên, giá trị mất mát trên tập kiểm thử (val) dao động nhẹ và không giảm đều, đặc biệt có xu hướng tăng lên ở các epoch sau cùng, điều này có thể là do giá trị learning rate cao khiến cho mô hình không tiến dần tới điểm cực tiểu.

Thứ hai, xét biểu đồ độ chính xác (Accuracy), ta nhận thấy độ chính xác trên tập huấn luyện liên tục tăng và dần tiệm cận giá trị 1, cho thấy mô hình đang học hiệu quả từ dữ liệu. Tuy nhiên, độ chính xác trên tập kiểm thử ổn định trong khoảng 85% - 90%, nhưng không tăng đều như trên tập huấn luyện. Sự chênh lệch giữa hai đường (train và val) trong biểu đồ độ chính xác cũng phản ánh rằng mô hình có thể gặp khó khăn trong việc tổng quát hóa tốt trên dữ liệu chưa từng thấy.

3.4 Mạng nơ-ron sử dụng khối (VGG)

3.4.1 Giới thiệu

AlexNet đã đánh dấu một bước ngoặt quan trọng trong lịch sử phát triển của mạng nơ-ron tích chập (CNN) và mở ra kỷ nguyên mới cho học sâu trong thị giác máy tính. Tuy nhiên, AlexNet cũng để lại một số hạn chế về mặt kiến trúc, đặc biệt là trong việc mở rộng và tối ưu hóa cấu trúc mạng. Sự thành công của AlexNet vào năm 2012 đã trở thành nguồn cảm hứng mạnh mẽ cho cộng đồng nghiên cứu, thúc đẩy sự phát triển của những kiến trúc mạng tốt hơn, đơn giản hơn nhưng mang lại hiệu quả cao hơn.

Vào năm 2014, Karen Simonyan và Andrew Zisserman, hai nhà nghiên cứu từ nhóm Visual Geometry Group (VGG) thuộc Đại học Oxford, đã công bố bài báo khoa học mang tính đột phá có tiêu đề “Very

Deep Convolutional Networks for Large-Scale Image Recognition". Bài báo này giới thiệu một loạt các kiến trúc mạng nơ-ron sâu được thiết kế với mục tiêu đạt hiệu suất cao hơn trong các bài toán thị giác máy tính. Nhóm nghiên cứu đã tiến hành thử nghiệm nhiều kiến trúc với độ sâu khác nhau, từ 11 tầng đến 19 tầng, và nhận thấy rằng việc tăng độ sâu của mạng giúp cải thiện đáng kể khả năng học và trích xuất đặc trưng. Điểm đặc biệt trong thiết kế của VGG là việc sử dụng các bộ lọc tích chập nhỏ kích thước 3x3 trên toàn bộ mạng, thay vì các bộ lọc lớn như trong AlexNet. Đồng thời, mạng cũng duy trì cấu trúc đồng nhất, dễ hiểu và dễ triển khai. Với việc sử dụng sức mạnh tính toán của các GPU tiên tiến, VGG được huấn luyện trên tập dữ liệu ImageNet quy mô lớn, bao gồm hàng triệu hình ảnh thuộc 1.000 lớp khác nhau. Trong cuộc thi ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2014, VGG đạt được thành tích xuất sắc, đứng vị trí thứ hai, chỉ sau GoogleNet (Inception).

Sự xuất hiện của VGG đã đặt nền móng cho sự phát triển của các kiến trúc mạng nơ-ron tiên tiến hơn như ResNet, Inception, và DenseNet. Các kiến trúc này tiếp tục tối ưu hóa độ sâu, giảm thiểu số lượng tham số, và cải thiện khả năng học đặc trưng của mạng. VGG không chỉ góp phần mở rộng phạm vi ứng dụng của CNN mà còn chứng minh rằng việc xây dựng các kiến trúc mạng sâu hơn, đơn giản hơn nhưng hiệu quả hơn là hướng đi đúng đắn cho sự phát triển lâu dài của học sâu. Với những đóng góp quan trọng, VGG đã trở thành một biểu tượng trong lịch sử nghiên cứu trí tuệ nhân tạo và tiếp tục ảnh hưởng mạnh mẽ đến các thế hệ mô hình sau này.

3.4.2 Kiến trúc VGG

Điểm nổi bật nhất của VGG-16 chính là chiều sâu vượt trội với 13 tầng tích chập hai chiều và 3 tầng fully connected, gấp nhiều lần so với AlexNet. Điều này giúp mô hình có khả năng học được những đặc trưng phức tạp và chi tiết hơn, đáp ứng tốt các yêu cầu xử lý dữ liệu hình ảnh ngày càng cao.



Hình 3.9: Kiến trúc mô hình VGG-16

Một trong những đóng góp nổi bật của VGG-16 là khái niệm về "khối tích chập"(Convolutional Block). Thay vì chỉ xen kẽ một tầng tích chập và một tầng max pooling, VGG-16 nhóm các tầng tích chập lại với nhau trước khi áp dụng max pooling. Cách tổ chức này không chỉ cải thiện hiệu quả trích xuất đặc trưng mà còn tạo nên một kiến trúc chuẩn hóa, đặt nền tảng cho các mạng CNN hiện đại.

VGG-16 cũng kế thừa hàm kích hoạt ReLU từ AlexNet, giúp giảm thiểu vấn đề gradient biến mất và tăng tốc độ huấn luyện. Một cải tiến quan trọng khác là việc sử dụng duy nhất các bộ lọc kích thước nhỏ 3x3, thay vì nhiều kích thước khác nhau như AlexNet. Sự lựa chọn này không chỉ giảm số lượng tham số mà còn tối ưu hóa hiệu quả tính toán. Ví dụ, hai bộ lọc 3x3 yêu cầu số tham số ít hơn so với một bộ lọc 5x5, nhưng vẫn mang lại khả năng trích xuất đặc trưng tương đương hoặc thậm chí tốt hơn.

Ngoài phiên bản VGG-16, còn có phiên bản mở rộng VGG-19, với 19 tầng tích chập, càng củng cố thêm hiệu quả của việc tăng chiều sâu trong mô hình. Từ đây, một hình mẫu chung cho các mạng CNN trong xử lý ảnh đã được thiết lập: sử dụng các khối tích chập với cấu trúc lặp lại, kết hợp các tầng max pooling để tối ưu hóa khả năng học đặc trưng. VGG-16 không chỉ là một thành tựu công nghệ mà còn là một bước tiến lớn, định hình tương lai của các mạng nơ-ron sâu.

3.4.3 Huấn luyện mô hình

a. Chuẩn bị dữ liệu viêm phổi, covid

```
# Download dataset
import opendatasets as opd
opd.download(
"https://www.kaggle.com/datasets/amanullahasraf/covid19-pneumonia-normal-chest-xraypa-
dataset?select=COVID19_Pneumonia_Normal_Chest_Xray_PA_Dataset")
```

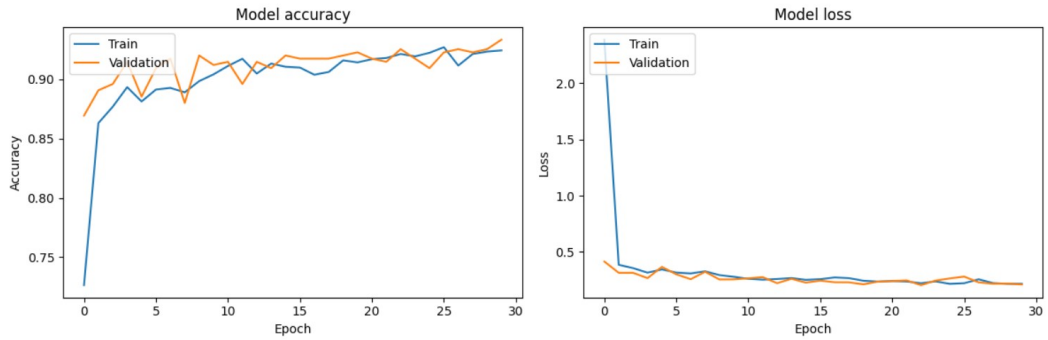
Trong bài này, chúng ta sẽ huấn luyện mô hình VGG-16 với tập dữ liệu COVID-19, Pneumonia Normal Chest X-ray PA. Bộ dữ liệu này bao gồm các hình ảnh X-quang ngực của bệnh nhân mắc COVID-19, viêm phổi và bình thường. Dữ liệu bao gồm các hình ảnh X-quang có kích thước khác nhau, với các nhãn chỉ ra tình trạng của bệnh nhân, như COVID-19, Pneumonia (viêm phổi), hoặc Normal (bình thường). Bộ dữ liệu này thường được sử dụng để thử nghiệm các mô hình phân loại hình ảnh trong các bài toán y tế, đặc biệt là trong việc nhận diện và phân loại các bệnh lý liên quan đến phổi từ X-quang ngực.

b. Huấn luyện mô hình

```
# Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
# Train model
history = model.fit(
    train_generator,
    epochs=30,
    validation_data=val_generator,
)
```

Đoạn code trên thực hiện việc biên dịch và huấn luyện mô hình AlexNet với bộ tối ưu hóa Adam, hàm mất mát Categorical Crossentropy, và đo lường độ chính xác (accuracy) trong quá trình huấn luyện. Mô hình được biên dịch với bộ tối ưu hóa Adam, một thuật toán phổ biến và hiệu quả để cập nhật trọng số của mạng nơ-ron, cùng hàm mất mát Categorical Crossentropy, phù hợp cho bài toán phân loại nhiều lớp. Chỉ số đo lường accuracy được sử dụng để theo dõi hiệu suất của mô hình trong suốt quá trình huấn luyện và kiểm tra. Sau đó, mô hình được huấn luyện trên dữ liệu từ train_generator và kiểm tra hiệu suất trên tập val_generator trong 30 epoch. Hàm fit trả về một đối tượng lịch sử (history), chứa thông tin chi tiết về độ chính xác và mất mát trên cả tập huấn luyện và tập xác thực trong từng epoch, giúp theo dõi sự cải thiện của mô hình qua từng chu kỳ huấn luyện.

c. Đánh giá mô hình



Hình 3.10: Quá trình huấn luyện mạng VGG

Đầu tiên, biểu đồ hàm mất mát (Loss) cho thấy rằng giá trị mất mát trên tập huấn luyện (Train) giảm dần qua từng epoch, minh chứng rằng mô hình dần dần học được các đặc trưng từ dữ liệu. Trong khi đó, giá trị mất mát trên tập kiểm thử (Validation) dao động nhẹ và có xu hướng ổn định ở các epoch sau. Thứ hai, biểu đồ độ chính xác (Accuracy) cho thấy độ chính xác trên tập huấn luyện liên tục tăng qua các epoch, chứng tỏ mô hình học được các đặc trưng của dữ liệu một cách hiệu quả. Độ chính xác trên tập kiểm thử cũng tăng nhưng không đều, có xu hướng dao động nhẹ ở một số epoch. Điều này cho thấy mô hình hoạt động khá tốt trên dữ liệu kiểm thử, tuy nhiên sự khác biệt giữa hai đường (Train và Validation) có thể là dấu hiệu cho thấy mô hình cần cải thiện khả năng tổng quát hóa.

3.5 Bài tập

Bài 1

Lập bảng so sánh sự khác biệt về kiến trúc mạng, số lớp, số lượng tham số, kích thước ảnh đầu vào, hàm kích hoạt giữa ba mô hình LeNet, AlexNet, VGG-16.

Bài 2

Tính lại số lượng parameter, output size của convolutional layer với stride và padding của mô hình LeNet với kích thước ảnh đầu vào là 28x28x1.

Bài 3

Tính lại số lượng parameter, output size của convolutional layer với stride và padding của mô hình AlexNet với kích thước ảnh đầu vào là 28x28x1.

Bài 4

Tính lại số lượng parameter, output size của convolutional layer với stride và padding của mô hình VGG-16 với kích thước ảnh đầu vào là 224x224x3.

Bài 5

Tại sao trong VGG-16, ở layer càng sâu thì width, height giảm nhưng depth lại tăng.

Bài 6

Xây dựng mô hình LeNet để huấn luyện trên dataset chữ cái (A-Z) của EMNIST.

Bài 7

Sử dụng mô hình LeNet đã train với dataset chữ cái và chữ số để nhận diện chữ số trên biển số xe.

Bài 8

Xây dựng mô hình AlexNet để huấn luyện trên dataset CIFAR-10.

Bài 9

Sử dụng/xây dựng mô hình VGG-16 để huấn luyện trên dataset Dog and Cat.

Bài 10

Tinh chỉnh lớp fully-connected của mô hình VGG-16 và so sánh khi mô hình chưa tinh chỉnh.